DELTA DESIGN & IMPLEMENTATION REPORT

Wishlist — Multi-Platform Price Tracking Application

Submitted	April 2026
Reviewer	S.A.Z
Stack	Python / FastAPI · React / Vite · Supabase · APScheduler
Decision	**Weak Accept → Targeting: Accept**

1. Introduction and Objectives

This report documents the architectural additions and refactoring tasks implemented during the final sprint, directly responding to feedback received in the formal design review. The work targeted the highest-priority gaps identified by the reviewer — specifically the non-functional notification pipeline, scheduler unreliability, and pattern deviations — while remaining within realistic time constraints ahead of Demo Day.

The original review decision was Weak Accept, citing a solid theoretical foundation undermined by incomplete execution. This delta addresses each core weakness methodically.

2. Review Feedback and Selection Justification

2.1 Critical Issues Identified in Review

#	Issue	Description	Severity
1	**Broken Price Alerting**	target_price existed in the DB but no mechanism notified the user when the threshold was crossed.	CRITICAL
2	**Scheduler Reliability**	Background polling crashed on Windows due to cp1254 charmap encoding faults; check_frequency field was never consumed.	HIGH
3	**UI Integration Gaps**	No notification visual system; price history graph had SVG distortion issues.	MEDIUM

2.2 Scope Selection and Cost Justification

Given the time constraints before Demo Day, entirely new core features (e.g., social login, multi-user sharing) were estimated at >3 days each and were deferred. The following scope was selected for maximum impact per effort:

Feature	Effort Estimate	Priority	Status
Observer-based Notification System & Scheduler Resilience	1 Day	**HIGH**	**COMPLETE**
Restoring Factory / Strategy Pattern Integrity	0.5 Days	**MEDIUM**	**COMPLETE**
UI: NotificationBell + Distortion-free Price Graph	0.5 Days	MEDIUM	**COMPLETE**

3. Implementation Details and Architectural Updates

3.1 Observer Pattern — Notification Architecture

The missing alerting mechanism — the most critical gap identified in the review — was resolved through a full Observer Design Pattern implementation.

Design Decision:

- Completely separated the act of fetching prices from the act of notifying users, eliminating the tight coupling that previously made notifications impossible to add cleanly.

Implementation:

- **PriceSubject** interface and its concrete implementation PriceMonitor.
- **check_prices** worker inside APScheduler iterates tracked products. On an active URL price change, PriceMonitor triggers .notify().
- **TargetPriceNotificationObserver** verifies new_price <= target_price and injects a structured alert into the notifications PostgreSQL table.

3.2 Dynamic Scheduling and System Resilience

The monolithic hourly task wasted resources for weekly-checked items and failed entirely when Windows Terminal codec issues were present.

Design Decision:

- Intelligent frequency-aware scheduling: skip items that have not yet reached their check_frequency threshold rather than blindly polling every hour.

Implementation:

- Overhauled the check_prices execution loop to perform a datetime diff against price_history.checked_at — items are skipped gracefully if their threshold has not elapsed.
- Attached next_run_time=datetime.now() to start_scheduler() so price backlogs are processed immediately when the backend comes online.

- Resolved cp1254 charmap encode exceptions that were silently destroying the scheduler loop in Windows Terminal environments.

3.3 Factory and Strategy Pattern Standardization

Deviations from the established architectural patterns were reverted and locked in.

Implementation:

- TrendyolScraper and HepsiburadaScraper are now firmly locked into the Strategy Pattern under a common interface.
- ScraperFactory provisions runtime extractors based on URL domains via _DOMAIN_REGISTRY, defaulting to LLMScraper for generic domains — restoring the genuine fallback capability stripped in a prior sprint.

3.4 UI/UX Enhancements

Implementation:

- NotificationBell component now uses frontend async polling to pull alert records dynamically, with real-time Mark as Read via PUT mapping.
- Dynamic Price Graphing: A unified scaling algorithm (chartMin / chartMax matrices) embeds the target_price threshold, ensuring the tracking view scales adaptively and eliminates SVG distortion through mathematically layered absolute HTML tracking nodes.

4. Impact on Quality Metrics and Testing

Quality Dimension	Impact	Result
Modularity / Coupling	PriceMonitor (Subject) and TargetPriceNotificationObserver (Observer) decouple notification logic from the Scheduler entirely.	IMPROVED
Efficiency	Frequency-aware scheduling eliminates unnecessary HTTP hits for weekly-tracked items that were previously polled every hour.	IMPROVED
Reliability	Codec bug fix prevents the daemon from silently failing; the scheduler no longer crashes on Windows Terminal environments.	FIXED
Design Conformance	CategoryFactory restoration and ScraperFactory pattern lock-in resolve the reviewer's architectural deviation findings.	RESOLVED
Automated Testing	Updated unit tests across test_scraper_factory.py, test_patterns.py, and test_category_factory.py; added pattern-driven tests for architecture stability.	UPDATED

5. Review Issue Resolution Map

The table below maps every weakness and missing mechanism flagged in the original review to the delta work that addresses it.

Original Review Finding	Delta Implementation	Status
Observer / Notification mechanism absent; target_price not wired to any logic	Full Observer Pattern: PriceMonitor + TargetPriceNotificationObserver + notifications table	RESOLVED
check_frequency field ignored; scheduler checks all products hourly	datetime diff loop in check_prices; items skipped until threshold elapsed	RESOLVED
cp1254 charmap exception silently destroying scheduler on Windows	Encoding fault resolved; daemon no longer fails silently	RESOLVED
LLMScraper fallback stripped; Strategy pattern integrity weakened	LLMScraper re-integrated as genuine fallback in ScraperFactory registry	RESOLVED
No notification UI; SVG price graph distortion	NotificationBell with async polling + chartMin/chartMax scaling algorithm	RESOLVED
Retry / Circuit Breaker absent (out of scope)	Deferred — exceeds Demo Day time budget	DEFERRED
Real DI Container (FastAPI Depends) absent (out of scope)	Deferred — exceeds Demo Day time budget	DEFERRED

6. Software Quality Assurance Plan (SQAP)

7.1 Purpose and Scope

This Software Quality Assurance Plan (SQAP) defines the activities, standards, and responsibilities used to ensure the quality of the Wishlist and Price Tracking Application throughout its development lifecycle. It covers all software artefacts produced by the project team, including source code, design documents, test suites, and deployment configurations. The plan is aligned with ISO/IEC/IEEE 12207:2017 (Software Life Cycle Processes) and ISO/IEC 25010 (Software Product Quality).

7.2 Quality Objectives

The following measurable quality objectives apply to the project:
Test Coverage: Unit and integration tests must achieve a minimum of 80% line coverage for all backend modules.
Defect Density: The number of open critical or high-severity defects at the time of each release must be zero.
Code Review Coverage: Code changes are reviewed by team members via WhatsApp discussion before being merged into the main branch.
Build Stability: The main branch must contain only passing code; the full pytest suite is executed after every merge.
Scraping Accuracy: The price extraction subsystem must maintain at least 90% accuracy on supported platforms (Trendyol, Hepsiburada, Amazon, Boyner, MediaMarkt, Teknosa).

System Availability: The backend service must achieve 99% uptime during the active demonstration and evaluation period.

7.3 Applicable Standards

ISO/IEC/IEEE 12207:2017 - Systems and Software Engineering - Software Life Cycle Processes
ISO/IEC 25010:2011 - Systems and Software Quality Requirements and Evaluation (SQuaRE) - Quality model
PEP 8 - Python Style Guide (applied to all backend code)
Conventional Commits - Commit message convention (applied to all Git commits)

7.4 SQA Activities

7.4.1 Code Review

Feature development is carried out on dedicated branches. Before merging into the main branch, the implementing team member shares the changes via WhatsApp for team discussion and agreement. Minor fixes are committed directly to main after verbal or written team consent.

7.4.2 Automated Testing

The project uses pytest for backend testing. Tests are organized into the following categories:
Unit Tests: Validate individual functions and classes in isolation (e.g., ScraperFactory, CategoryService, PriceMonitor). 283 unit tests are currently passing with 0 failures.
Integration Tests: Validate interactions between components and the Supabase backend.
API Tests: Validate all REST endpoints for correct HTTP status codes, response schemas, and error handling.

7.4.3 Process Auditing

At the end of each sprint, the team reviews progress and process adherence via WhatsApp group communication, checking that branches are merged, tests pass, and documentation is up to date.

7.4.4 Defect and Issue Tracking

Bugs and defects identified during development or testing are reported in the team WhatsApp group. The team agrees on priority and the responsible member resolves the issue. Significant defects and their resolutions are documented in the shared Google Docs folder for traceability.

7.4.5 Static Analysis

Python backend code is checked with flake8 for style compliance and mypy for type correctness. Frontend React code is validated with ESLint. These checks are run locally before committing.

7.5 Responsibilities

All team members are responsible for writing unit tests for the code they produce before committing.
The committing team member is responsible for running the full test suite and ensuring it passes before merging.
Team members review each other's changes informally via WhatsApp before branch merges.
The project lead is responsible for coordinating sprint reviews and ensuring documents remain up to date.

7.6 Verification and Validation

Verification (Are we building the product right?): Each module is verified through unit and integration tests and peer discussion before being merged into the main branch.
Validation (Are we building the right product?): The system is validated against the functional requirements defined in the use case section during integration testing and manual end-to-end testing. Scraping accuracy is validated against a benchmark of 20 real product pages per supported platform.

7.7 Error and Change Management

Bug Reports: Bugs discovered during testing or operation are reported in the team WhatsApp group and documented in the shared Google Docs folder. Critical bugs affecting core functionality (price tracking, authentication) are prioritized and resolved in the next working session.
Change Requests: Scope changes or requirement modifications are discussed and agreed upon by all team members via WhatsApp before implementation. Approved changes are documented in the shared Google Docs folder and reflected in updated design and requirements documents.
Regression Policy: Whenever a bug fix or new feature is committed, the full pytest test suite is re-executed locally to ensure no regressions have been introduced.

7.8 Test Plans and Criteria

Entry Criteria for Testing: Code must pass static analysis (flake8, ESLint) and build without errors.
Exit Criteria for a Sprint: All planned features are implemented, all unit tests pass, and no critical defects remain open.
Acceptance Testing Criteria: All defined use cases must execute successfully in the staging environment. Scraping accuracy must meet or exceed 90% on supported platforms.
Performance Testing: Response time for REST API endpoints must be below 500 ms under normal load (up to 50 concurrent users).

7.9 Reporting and Record Keeping

Test Reports: Pytest generates a detailed test report (test_report.txt) after each test run, capturing pass/fail status for all 283 test cases.
Issue Log: Defects and change decisions are recorded in the shared Google Docs folder, with descriptions and resolution notes maintained by the responsible team member.
Review Records: Code changes are managed through feature branches; team members communicate feedback via WhatsApp before merging into the main branch. Direct pushes to main are used for minor fixes after team agreement.
Sprint Retrospective Notes: Team coordination and retrospective discussions are conducted via WhatsApp, with key decisions and task assignments documented in the shared Google Docs folder.
LLM Evaluation Report: The scraping accuracy evaluation results (docs/PA3/README.md) are maintained as part of the quality record, showing 68.3% overall field-level accuracy and identifying areas for improvement.

7. Conclusion

The delta sprint directly targeted the four findings that drove the Weak Accept decision in the original review. The Observer-based notification pipeline activates the application's core value proposition for the first time. Frequency-aware scheduling and the codec fix restore scheduler reliability. Reverting to CategoryFactory and re-integrating the LLMScraper fallback eliminates the pattern mislabeling and regression concerns. The price graph and NotificationBell complete the UI integration gaps.

All six resolvable issues from the review have been addressed. Two items — Retry/Circuit Breaker logic and a real DI container — remain deferred due to implementation cost versus the Demo Day timeline, and are recommended as the next iteration's first priorities.

The implementation now more fully realizes the design intent described in the original submission, and the project is positioned for an Accept outcome in a follow-up evaluation.

Task Matrix

Tasks	Akif Emre Reis	Alaaddin Gürsoy	Elif Serra Öncü	Gizem Elif Bayar
Introduction and Objectives			✓	✓
Review Feedback and Selection Justification	✓	✓		
Implementation Details and Architectural Updates		✓		✓
Impact on Quality Metrics and Testing	✓		✓	
Review Issue Resolution Map		✓	✓	

Conclusion	✓			✓